

Task 1: Develop a c program using fork() that creates a parent and child process. Display the process ID (PID), Parent process ID (PPID), and process states at different stages of execution

Step 1

Create file

```
srikar@DESKTOP-0NIMS2G: ~  
printf("PARENT PROCESS\n");  
printf("Parent PID: %d\n", getpid());  
printf("Child PID: %d\n", pid);  
  
printf("Parent waiting for child...\n");  
sleep(35);  
  
wait(NULL);  
  
printf("Child completed. Parent exiting.\n");  
}  
  
return 0;  
}srikar@DESKTOP-0NIMS2G:~$ gcc process_state.c -o process_state  
srikar@DESKTOP-0NIMS2G:~$ ./process  
-bash: ./process: No such file or directory  
srikar@DESKTOP-0NIMS2G:~$ ./process_state  
PARENT PROCESS  
Parent PID: 11338  
Child PID: 11339  
Parent waiting for child...  
CHILD PROCESS  
Child PID: 11339  
Parent PID: 11338  
Child is RUNNING...  
Child is WAITING/SLEEPING...
```

Step 2

```
srikar@DESKTOP-0NIMS2G: ~  
printf("Parent waiting for child...\n");  
sleep(35);  
  
wait(NULL);  
  
printf("Child completed. Parent exiting.\n");  
}  
  
return 0;  
}srikar@DESKTOP-0NIMS2G:~$ gcc process_state.c -o process_state  
srikar@DESKTOP-0NIMS2G:~$ ./process  
-bash: ./process: No such file or directory  
srikar@DESKTOP-0NIMS2G:~$ ./process_state  
PARENT PROCESS  
Parent PID: 11338  
Child PID: 11339  
Parent waiting for child...  
CHILD PROCESS  
Child PID: 11339  
Parent PID: 11338  
Child is RUNNING...  
Child is WAITING/SLEEPING...  
Child is TERMINATING...  
Child completed. Parent exiting.  
srikar@DESKTOP-0NIMS2G:~$ ps -o pid,ppid,stat,cmd  
PID PPID STAT CMD  
4796 4792 Ss -bash  
11724 4796 R+ ps -o pid,ppid,stat,cmd  
srikar@DESKTOP-0NIMS2G:~$
```

Step 3

```
}  
  
    return 0;  
}  
  
rikar@DESKTOP-0NIMS2G:~$ gcc process_state.c  
rikar@DESKTOP-0NIMS2G:~$ ./process_state  
PARENT PROCESS  
Parent PID: 13171  
Child PID: 13172  
  
Parent waiting for child...  
  
CHILD PROCESS  
Child PID: 13172  
Parent PID: 13171  
  
Child is RUNNING...
```

Output

```
srikar@DESKTOP-ONIMS2G: ~  
Child PID: 13172  
  
Parent waiting for child...  
  
CHILD PROCESS  
Child PID: 13172  
Parent PID: 13171  
  
Child is RUNNING...  
  
Child is WAITING/SLEEPING...  
  
Child is TERMINATING...  
ps -o pid,ppid,stat,cmd | grep process_state  
  
Child completed. Parent exiting.  
srikar@DESKTOP-ONIMS2G:~$ ps -o pid,ppid,stat,cmd | grep process_state  
13404 4796 S+ grep --color=auto process_state  
srikar@DESKTOP-ONIMS2G:~$ ./process_state &  
[1] 13883  
srikar@DESKTOP-ONIMS2G:~$ PARENT PROCESS  
Parent PID: 13883  
Child PID: 13884  
  
Parent waiting for child...  
  
CHILD PROCESS  
Child PID: 13884  
Parent PID: 13883  
  
Child is RUNNING...  
ps -o pid,ppid,stat,cmd | grep '[p]rocess_state'  
Child is WAITING/SLEEPING...  
ps -o pid,ppid,stat,cmd | grep '[p]rocess_state'  
grep: pid,ppid,stat,cmd: No such file or directory  
srikar@DESKTOP-ONIMS2G:~$  
Child is TERMINATING...  
  
Child completed. Parent exiting.  
srikar@DESKTOP-ONIMS2G:~$
```

Task 2 Design an experiment to observe process state transitions (Ready, Running, Waiting, terminates) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Step 1 Create file

```
imvarls@LAPTOP-VQ4D70L7: ~$ nano demo.c
```

Step 2 Code

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();
    if (pid == 0) {
        // Child process
        printf("Child PID: %d\n", getpid());
        sleep(5); // Waiting state
        for (int i = 0; i < 1e8; i++); // Running state
        exit(0); // Terminated
    } else {
        // Parent process
        printf("Parent PID: %d\n", getpid());
        sleep(10); // Keep parent alive to observe child
    }
    return 0;
}
```

Outputs

```
srikar@DESKTOP-0NIMS2G: ~
printf("Parent waiting for child...\n");
sleep(35);

wait(NULL);

printf("Child completed. Parent exiting.\n");
}

return 0;
}
srikar@DESKTOP-0NIMS2G:~$ gcc process_state.c -o process_state
srikar@DESKTOP-0NIMS2G:~$ ./process
-bash: ./process: No such file or directory
srikar@DESKTOP-0NIMS2G:~$ ./process_state
PARENT PROCESS
Parent PID: 11338
Child PID: 11339
Parent waiting for child...
CHILD PROCESS
Child PID: 11339
Parent PID: 11338
Child is RUNNING...
Child is WAITING/SLEEPING...
Child is TERMINATING...
Child completed. Parent exiting.
srikar@DESKTOP-0NIMS2G:~$ ps -o pid,ppid,stat,cmd
  PID   PPID  STAT CMD
  4796   4792  Ss   -bash
 11724   4796  R+    ps -o pid,ppid,stat,cmd
srikar@DESKTOP-0NIMS2G:~$
```

```

top - 03:59:36 up 7 min, 1 user, load average: 0.00, 0.01, 0.00
Tasks: 25 total, 1 running, 24 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.0 sy, 0.0 ni, 100.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 7782.6 total, 6493.1 free, 540.0 used, 901.7 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 7242.7 avail Mem

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	24232	15376	11584	S	0.0	0.2	0:00.75	systemd
2	root	20	0	3180	2212	2072	S	0.0	0.0	0:00.01	init-systemd(Ub
7	root	20	0	3180	2044	1940	S	0.0	0.0	0:00.00	init
47	root	19	-1	50428	16684	15424	S	0.0	0.2	0:00.17	systemd-journal
79	systemd+	20	0	22540	14616	11996	S	0.0	0.2	0:00.08	systemd-resolve
85	root	20	0	35364	12352	9260	S	0.0	0.2	0:00.32	systemd-udevd
144	root	20	0	2888	1948	1820	S	0.0	0.0	0:00.06	chronyd-starter
145	root	20	0	4472	3160	2908	S	0.0	0.0	0:00.01	cron
146	message+	20	0	8800	5508	4828	S	0.0	0.1	0:00.07	dbus-daemon
153	root	20	0	44092	29704	14564	S	0.0	0.4	0:00.38	networkd-dispat
157	root	20	0	18432	9404	8160	S	0.0	0.1	0:00.11	systemd-logind
160	root	20	0	1866288	15164	12552	S	0.0	0.2	0:00.13	wsd-pro-service
183	syslog	20	0	220548	5908	4632	S	0.0	0.1	0:00.12	rsyslogd
214	_chrony	20	0	24448	11980	7216	S	0.0	0.2	0:00.10	chronyd
219	root	20	0	5492	2724	2540	S	0.0	0.0	0:00.02	agetty
223	root	20	0	123456	32592	17980	S	0.0	0.4	0:00.28	unattended-upgr
232	_chrony	20	0	12092	2444	1796	S	0.0	0.0	0:00.00	chronyd
330	root	20	0	8644	5556	4716	S	0.0	0.1	0:00.01	login
372	imvarls	20	0	22332	13516	10900	S	0.0	0.2	0:00.09	systemd
376	imvarls	20	0	22912	4092	2088	S	0.0	0.1	0:00.00	(sd-pam)
402	imvarls	20	0	6136	5480	3928	S	0.0	0.1	0:00.01	bash
575	root	20	0	3184	1124	980	S	0.0	0.0	0:00.00	SessionLeader
576	root	20	0	3200	1260	1108	S	0.0	0.0	0:00.01	Relay(581)
581	imvarls	20	0	6268	5536	3864	S	0.0	0.1	0:00.04	bash
699	imvarls	20	0	8296	6056	3888	R	0.0	0.1	0:00.01	top